

How the Workflow Engine Will "Think"

wrkflw · engine-core design discussion · 2026-06-20 · written so a junior developer can learn from it

This document explains **three ways** we could build the heart of our workflow engine, and why one of them is recommended. It assumes **no prior knowledge** of workflow engines. If a word looks unfamiliar, check the [Glossary](#) at the end — every technical term is defined there in plain language.

1. [What is a workflow engine?](#)
2. [The core idea: tokens](#)
3. [The decision we're making](#)
4. [Option A — Stepper + commands](#)
5. [Option B — Active engine](#)
6. [Option C — Hybrid](#)
7. [How a result comes back](#)
8. [Side-by-side comparison](#)
9. [Controlling time \(clockwork\)](#)
10. [Recommendation](#)
11. [Glossary](#)

1. What is a workflow engine?

Imagine a business process: a customer submits a leave request, a manager approves it, HR records it, and an email is sent. That sequence of steps — some automatic, some needing a human, some with deadlines — is a **workflow**. A **workflow engine** is software that *runs* these processes: it knows which step comes next, waits when it must wait, calls the right code at the right time, and remembers exactly where every running process is, even if the server restarts.

Analogy — a board game. Think of a process as a board with squares connected by arrows. A **process definition** is the printed board (the rules and layout). A **process instance** is one actual play-through of the game. A **token** is your game piece sitting on a square. The engine's whole job is to decide, for each piece, *which arrow to follow next* and *when*. Some squares say "wait for a human to roll the dice" (a human task); some say "wait 3 days" (a timer); some split your piece into several pieces that move down different paths at once (a parallel gateway).

2. The core idea: tokens moving on a graph

Our engine is **token-based**. A token is a marker that sits on one node (square) and carries a bag of **variables** (data about this run — who requested leave, how many days, etc.). When the engine moves a token along an arrow (a **sequence flow**) to the next node, the node may read those variables to decide what to do. For example, an **exclusive gateway** ("either/or" fork) might check `days > 10` and send the token down the "needs director approval" path instead

of the normal path. We evaluate those little conditions with a library called [expr](#), so we don't invent our own mini-language.

This whole project follows **BPMN** — an industry-standard visual notation for business processes — so our nodes have familiar names: tasks, gateways, events, and the flows between them. The full list of terms is in the [Glossary](#).

3. The decision we're making

The most important rule for this part of the system: the **engine core must be "pure"**. That means the core logic — deciding which token moves where — must **not** directly talk to the database, the network, the message bus, or the clock. Why? Because keeping it pure makes it:

- **Easy to test** — you can check its decisions without setting up a database or server.
- **Easy to reuse** — a company can embed our engine in their app without dragging in our infrastructure choices.
- **Reliable** — pure logic behaves the same way every time, which makes crash-recovery and debugging far easier.

So the question is: **if the core can't do input/output itself, how does it actually get work done?** (Calling a service, scheduling a timer, sending an email.) Below are three answers. The differences are entirely about *who performs the side effects* and *how a result gets back into the engine*.

4. Option A — A "stepper" that returns a to-do list RECOMMENDED

The core is one **pure function** called `Step`. You give it the current situation and one thing that just happened (a **trigger**). It hands back the new situation plus a **to-do list of commands** — descriptions, written as plain data, of side effects it wants someone else to perform. The core never does them itself.

Analogy — a chef who only writes tickets. The core is a head chef who never touches a pan. You tell the chef "a new order arrived" (a trigger). The chef thinks, updates the order board, and writes tickets: "fry this," "set a 5-minute timer," "tell the waiter." The chef hands those tickets to the kitchen staff (the **runtime**). When a dish is done, the staff bring back a "ticket #7 finished" note (a result), and the chef thinks again. The chef is fast, predictable, and never burns anything — because the chef only ever *decides*.

```
// The ENTIRE job of the core: take a situation + one event, return the
// new situation + a list of side effects to perform. No database. No network.
func Step(def *ProcessDefinition, state InstanceState, stim Trigger) (StepResult, error)

type StepResult struct {
    State      InstanceState // the new state — someone else will save this to the data
    Commands []Command           // the "to-do list" — plain data, not code to run
}

// A "trigger" is something that happened the engine must react to:
//   StartInstance, ActionCompleted, TimerFired, HumanCompleted, SignalReceived ...
// A "command" is something the engine wants done:
//   InvokeAction, ScheduleTimer, EmitEvent, AwaitHuman, Compensate ...
```

WHY JUNIORS WILL LIKE IT

- Testing is "call function, check the returned to-do list." No mocks, no servers.
- Each piece does one clear job, so it's easy to read.
- "Waiting for a human for 3 days" needs zero memory — the process is simply saved and woken later.

THE TRADE-OFF

- Someone must run the "kitchen staff" loop that performs the commands. We'll ship a default one.
- We must design the fixed list of trigger/command types up front.

5. Option B — An "active" engine that does everything itself

DISCOURAGED

The core is handed tools (database, scheduler, message bus) and **does the work directly**: it calls the service, starts the timer, sends the event, and manages its own background threads as it walks the graph.

Analogy — a chef who also cooks, waits tables, and answers the phone. Simpler to set up for one small order, but the chef is now juggling everything at once. Hard to test (you must fake the whole kitchen), and hard to recover if the chef faints mid-service — you can't tell exactly what was finished.

UPSIDE

- Easiest to call for a tiny embedded use: "just run it."
- Fewer parts to wire on the happy path.

WHY WE LEAN AWAY

- Mixes background threads and input/output *into* the part we most need to keep pure.
- Crash-recovery and "replay to debug" become hard.
- Hard to guarantee events and saved state stay in sync (the outbox problem).

6. Option C — A hybrid: pure decisions, but it calls a few helpers

FALLBACK

A middle path. The core decides purely (no background threads), but *during* a step it calls a small number of injected helper functions (call this service, emit this event, read the clock) and gets results back immediately, instead of returning a to-do list.

UPSIDE

- No background threads inside the core (better than B).
- Results come back right away — less bookkeeping than A.

DOWNSIDE

- The core still does input/output mid-decision, so clean replay and the outbox are weaker than A.
- Long "wait 3 days for a human" doesn't fit an immediate function call neatly.

7. How a result comes back (walking through Option A)

This is the part that confuses people at first, so let's go slowly. The trick is one simple rule:

The rule: every command that will produce a result is stamped with an **CommandID** (a unique label). When the result is ready, it comes back carrying *that same CommandID*. That label is how the engine knows "ah, this is the answer to the thing I asked for earlier."

- 1 Something happens (e.g. a new request). The runtime calls `Step(state, trigger)`.
- 2 The core decides, returns the **new state** and a **to-do list**, e.g.
`InvokeAction{CommandID:"7", name:"checkCredit"}`.
- 3 The runtime **saves the new state** to the database (the database is the single source of truth).
- 4 The runtime **performs each command**: it actually calls the `checkCredit` service.
- 5 When `checkCredit` finishes, the runtime creates a new trigger
`ActionCompleted{CommandID:"7", output:...}` — **same label, 7.**
- 6 The runtime calls `Step` again with that trigger. The core finds the token waiting on "7", merges the output, and moves on.

```
// The "kitchen staff" loop. NOTE: this lives OUTSIDE the core, in a later
// sub-project. The pure core never contains this loop.
for {
    res, _ := core.Step(def, state, stim)           // 1-2. pure decision + to-do list
    save(res.State)                                 // 3. database is the source of truth
    for _, e := range res.Commands {                // 4. actually do the side effects
        switch e := e.(type) {
        case InvokeAction:
            out, _ := catalog.Resolve(e.ActionName).Do(ctx, e.Input)
            enqueue(ActionCompleted{CommandID: e.CommandID, Output: out}) // 5. same label
        case ScheduleTimer:
            scheduler.At(e.FireAt, func(){ enqueue(TimerFired{CommandID: e.CommandID})})
        case AwaitHuman:
            // nothing to do now – the process simply waits in the database
            // until a person completes the task via the API later.
        }
    }
    stim = dequeue()                               // 6. wait for the next thing to happen,
}
```

Why this is powerful: "wait 3 working days for a manager, send a reminder on day 2, and if nobody acts, take an alternative path" becomes just a few commands: `AwaitHuman` + a reminder `ScheduleTimer` + an SLA `ScheduleTimer`. Then the process is **saved and forgotten** — using no memory or threads — until a timer fires or the human acts. That single idea covers human tasks, timers, SLAs, retries, and even rollback (a `Compensate` command), which is exactly what our requirements ask for.

8. Side-by-side comparison

● strong ● okay, with effort ● weak

What we care about	A · Stepper + commands	B · Active engine	C · Hybrid
Keeps the core pure (no DB/network/threads)	● yes	● no	● mostly
Easy to test without mocks	● yes	● hard	● some
Keeps events & saved state in sync (outbox)	● natural	● hard	● awkward
Recover after a crash / replay to debug	● yes	● hard	● partial
Long waits (human / 3-day SLA) cost nothing	● yes	● managed	● clumsy
Rollback / compensation fits cleanly	● yes	● manual	● manual
Simplest to call for a tiny demo	● needs loop	● easiest	● easy
Up-front design effort	● medium	● low	● low

9. Controlling time without waiting (clockwork)

Timers and deadlines are everywhere in workflows, but tests can't really wait 3 days. We use a library called `clockwork`, which gives us a **fake clock** we can fast-forward in tests. Our scheduler library (`gocron`) understands the same fake clock, so in a test we can jump "3 days forward" instantly and watch the SLA logic fire — in real time it would genuinely wait 3 days.

In **Option A**, the core never even looks at the clock — it just says "schedule a timer for this moment," and the runtime resolves the actual time using the shared clock. That is the cleanest of the three and the easiest to test.

10. Recommendation

Go with Option A (stepper + commands as data). It is the only option that truly keeps the engine's brain pure, and it makes several hard requirements — keeping events in sync, retrying safely, rolling back, and waiting days for a human — almost *fall out for free*. The one cost (a runtime loop instead of a single "just run it" call) is paid once, later, and we'll ship a ready-made loop so simple uses stay easy.

If the team decides day-one simplicity matters more than crash-recovery and event safety, **Option C** is the backup. **Option B** is discouraged because it puts threads and input/output inside the very part we need to keep clean.

11. Glossary (plain-language)

Workflow / Process

A sequence of steps that accomplishes a business goal (e.g. approving leave).

Workflow engine

Software that runs processes: tracks where each one is, runs steps, waits, and remembers state across restarts.

BPMN

"Business Process Model and Notation" — an industry-standard visual language for drawing processes (tasks, gateways, events, arrows). We model our concepts on it.

Process definition

The reusable template (the "board"). Loaded from YAML, Go code, or BPMN2 XML.

Process instance

One running execution of a definition (one "play-through").

Node

A point in the process: a task, a gateway, or an event.

Sequence flow

An arrow connecting two nodes — the path a token can travel.

Token

A marker sitting on a node that represents "the process is here," carrying the run's variables.

Variables

The data a process instance carries (who, how many days, amounts, etc.), read by gateways to make decisions.

Gateway

A branching node. **Exclusive** = pick one path (either/or). **Parallel** = take all paths at once and later re-join. **Inclusive** = pick one or more.

Event

Something that happens to the process — e.g. a **timer** firing, or a **signal** arriving.

Service action

A reusable piece of code (referenced by name in the definition) that does real work — call an API, send an email. All actions share one Go interface.

Human task

A step that waits for a person to act (approve, fill a form), often with a deadline.

SLA (deadline)

"Service-Level Agreement" — a promised time limit (e.g. 3 working days). If breached, the process takes an alternative action/path.

Trigger

(Option A) A piece of data saying "this just happened" that we feed into the core — e.g. an action finished, a timer fired, a human responded.

Command

(Option A) A piece of data saying "please do this" that the core returns — e.g. call an action, schedule a timer, emit an event.

CommandID

(Option A) A unique label on a command; the matching result carries the same label so the engine can correlate them.

Runtime / loop

(Option A) The code outside the core that performs commands and feeds results back in. The core itself does not contain it.

Pure function

A function whose output depends only on its inputs and that causes no side effects (no database, no network). Predictable and easy to test.

Side effect

Anything that touches the outside world: writing to a database, calling a service, sending a message.

Expression (expr)

A small condition like `days > 10`, evaluated by the expr library to make routing decisions.

Outbox pattern

A reliability technique: save state and the "to-publish" events in the *same* database transaction, then publish the events afterward — so they can never disagree.

Compensation / rollback

Undoing earlier steps when something goes wrong (e.g. refund a charge). Each node may have an optional compensation action.

Idempotency

Doing the same operation twice has the same effect as doing it once — important when work might be retried after a crash.

clockwork

A Go library providing a fake, fast-forwardable clock so time-based logic can be tested instantly.

gocron

The Go scheduling library we use (pinned to v2.21.2) for timers, SLA waiters, and in-wait actions.

wrkflw — engine-core execution-model explainer · discussion artifact, not a final spec · saved at <docs/specs/2026-06-20-engine-core-execution-model-explained.html> (PDF rendered alongside). Once the team chooses an option, it becomes an ADR and is folded into the engine-core design spec.